

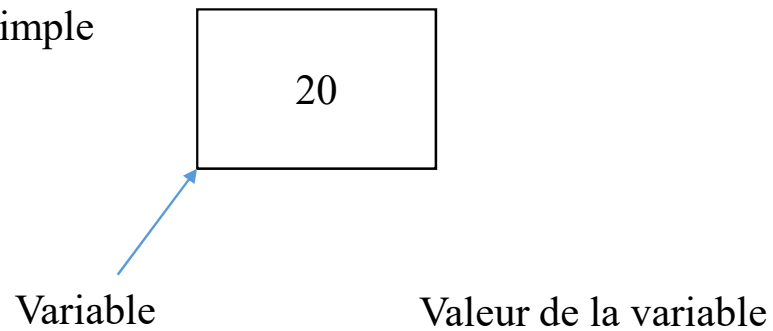
# Chapitre 5 : Tableaux

# Tableaux

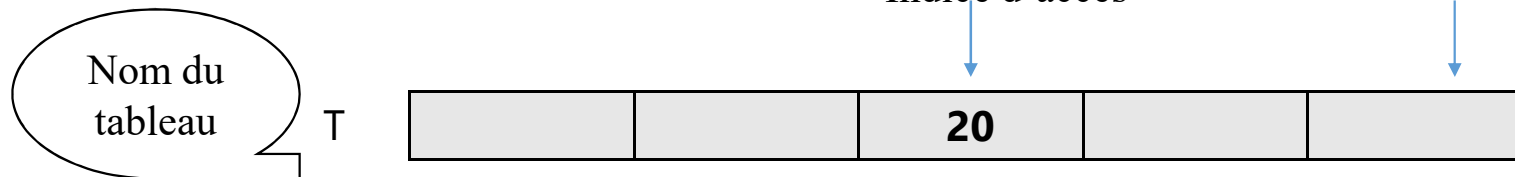
- Un tableau est une structure de donnée  $T$  qui permet de stocker un certain nombre d'éléments  $T[i]$  repérés par un index  $i$
- Tableau= collection de données homogènes, accessibles par un indice entier
- Tous les éléments dans un tableau ont le même type de base
- Le nombre d'éléments stockés dans un tableau est fixé
- Un tableau est un vecteur de dimension  $N$

# Tableaux

- Pour une variable simple



- Pour un tableau



# Tableaux

- Déclaration d'un tableau:  
Nom\_Tableau: tableau [1..N] de **Type**

Avec Type est le type des éléments du tableau

## Exemple

T1: Tableau [1..100] d'entier

T2: Tableau [1..20] de réel

T3: Tableau [1..50] de caractère

# Tableaux

- Accès aux composantes d'un tableau: Pour accéder à un élément, il suffit de préciser entre crochets l'indice de la case contenant cet élément
- On a un tableau T de dimension N
  - L'accès au premier élément du tableau se fait par T[1]
  - L'accès au dernier élément du tableau se fait par T[N]

|         | 100  | 200  | 300  | 400  | 500  |
|---------|------|------|------|------|------|
| Indice  | 1    | 2    | 3    | 4    | 5    |
| Contenu | T[1] | T[2] | T[3] | T[4] | T[5] |

# Tableaux

- Chargement d'un tableau :

```
AlgorithmeRemplirTab1D
Constantes
    N=5
Variables
    T : Tableau[1..N] de réel
    i : entier
Début
    Pour i de 1 à N faire
        Écrire("Donner un nombre : ")
        Lire(T[i])
    FinPour
Fin
```

- Affichage du contenu d'un tableau :

```
AlgorithmeAfficherTab1D
Constantes
    N=5
Variables
    T : Tableau[1..N] de réel
    i : entier
Début
    Pour i de 1 à N faire
        Écrire(T[i])
    FinPour
Fin
```

- Un tableau doit être lu élément par élément et non pas comme une seule entité. Il sera alors interdit d'écrire : Lire(nom\_tab) 

# Tableaux

- Remarque

**Constante**

max=100

**Types**

Tab: Tableau [1..max] de entier

**Variables**

T : Tab

# Tableaux

```
Procédure Lecture(var tab : tableau [1..N] d'entiers, N : Entier)
Var
    i : Entier
Début
    Pour i de 1 à N faire
        Ecrire("Entrez la valeur ", i, " du tableau : ")
        Lire(tab[i])
    FinPour
FinProcédure
```

# Tableaux

```
Procédure Affichage(tab : tableau [1..N] d'entiers, n : Entier)
Var
    i : Entier
Début
    Pour i de 0 à N-1 faire
        Ecrire("tab[", i , "] = ", tab[i])
    FinPour
FinProcédure
```

# Tableaux

Les affectations s'appliquent aux tableaux comme aux variables:

- La variable x va prendre la valeur du premier élément du tableau :  **$X \leftarrow \text{Tab}[1]$**
- Le deuxième élément du tableau va prendre la valeur 50 :  **$\text{Tab}[2] \leftarrow 50$**

# Exercice

On veut déterminer s'il existe un indice  $i$  allant de 1 à  $N$  tel que  $val = T[i]$

## ■ Tableau non Ordonné

**Algorithme** *Recherche1*

**Var**  $i, val$  : **Entier**,  $T$ : **tableau**  $[1, N]$ : **Entier**,  $Trouve$ : **Booléen**  
**Début**

$Trouve \leftarrow$  Faux

**Pour**  $i$  **de** 1 **à**  $N$  **Faire**

**Si** (  $T[i]=val$  ) **alors**

$Trouve \leftarrow$  Vrai

**Fin Si**

**Fin Pour**

**Fin**

# Exercice

- Soit T un tableau contenant n éléments de type entier. Ecrire un algorithme MinTab qui affiche le plus petit élément de ce tableau.(on suppose que T est déjà rempli)

```
Algorithme MinTab
Constantes      N = 5
Variables       T : Tableau[1..N] de réel
                  min,i: entier

Début
    min ← T[1]
    Pour i de 2 à N Faire
        Si (T[i] < min) Alors
            min ← T[i]
        FinSi
    FinPour
    Ecrire(min)
Fin
```

# Exercice

## Suppression d'un élément d'un tableau

- Soit T un tableau d'entiers de N éléments. Écrire un algorithme permettant de supprimer le i<sup>ème</sup> élément du tableau T.

**Algorithme Suppression\_tab**  
**Var** T : tableau [1..50] d'entiers  
i, n, ind : entier

**Début**

**Répéter**

Lire (n)

**Jusqu'à** (n > 0 et n <= 50)

**pour** i de 1 à n **faire**

lire (T[i])

**fin pour**

**Répéter**

ecrire ("Donner l'indice de suppression : ")

Lire (ind)

**Jusqu'à** (ind > 0 et ind <= n)

// décalage à gauche d'une partie du tableau

**pour** i de ind à n-1 **faire**

T[i] ← T[i+1]

**fin pour**

// réduction de la taille du tableau

n ← n-1

// affichage après suppression

**pour** i de 1 à n **faire**

ecrire (T[i])

**fin pour**

**Fin**

# Exercice

## Insertion d'un élément dans un tableau

- Soit T un tableau d'entiers de N éléments. Soit x un entier donné. Écrire un algorithme permettant d'insérer x à un indice donné du tableau T puis d'afficher T.

**Algorithme Insertion\_tab**

**Var** T : tableau [1..50] d'entiers

i, n, x, ind : entier

**Début****Répéter**

Lire (n)

**Jusqu'à** (n > 0 et n < 50)

**pour** i de 1 à n **faire**

lire (T[i])

**fin pour**

ecrire ("Donner l'élément à insérer : ")

Lire (x)

ecrire ("Donner l'indice d'insertion : ")

**Répéter**

Lire (ind)

**Jusqu'à** (ind > 0 et ind ≤ n)

// extension de la taille du tableau

n ← n+1

// décalage à droite d'une partie du tableau

**pour** i de n à ind+1 [pas = -1] **faire**

T[i] ← T[i-1]

**fin pour**

// insertion de x

T[ind] ← x

// affichage

**pour** i de 1 à n **faire**

ecrire (T[i])

**fin pour**

**Fin**

# Exercice

## Recherche séquentielle d'un élément dans un tableau

Soit T un tableau contenant n éléments de type entier.

On veut écrire un algorithme qui cherche un entier x et qui affiche :

- L'indice de la première occurrence de x dans T si x appartient à T
- Le message "élément introuvable..." si x n'appartient pas à T

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
| <b>Algorithme Recherche</b>            | $i \leftarrow 0$                        |
| <b>Variables</b>                       | <b>Répéter</b>                          |
| T : Tableau[1..N] de entier            | $i \leftarrow i + 1$                    |
| x, i: entier                           | <b>Jusqu'à (T[i] = x) ou (i &gt; n)</b> |
| <b>Début</b>                           | <b>Si (T[i] = x) Alors</b>              |
| // on suppose que T est déjà rempli    | Ecrire("Indice = ", i)                  |
| Ecrire ("donner l'élément à chercher") | <b>Sinon</b>                            |
| Lire(x)                                | Ecrire("Elément introuvable...")        |
|                                        | <b>FinSi</b>                            |
|                                        | <b>Fin</b>                              |

# Méthodes de tri dans un tableau

- Trier un tableau c'est donc ranger les éléments d'un tableau en ordre croissant ou décroissant
- Il existe plusieurs méthodes de tri qui se différencient par leur complexité d'exécution et leur complexité de compréhension pour le programmeur
- On va voir trois types :
  - Tri à bulles
  - Tri par sélection
  - Tri par insertion

# Tri à bulles

Le tri à bulles consiste à faire **remonter progressivement les plus grands éléments** d'un tableau, comme les bulles d'air remontent à la surface de l'eau

## *Principe :*

- Parcourir les éléments du tableau de 1 à  $(n-1)$
- Si l'élément  $(i)$  est supérieur à l'élément  $(i+1)$ , on les permute
- Le programme s'arrête lorsqu'aucune permutation n'est réalisée après un parcours complet du tableau

# Tri à bulles

**Algorithme TriABulles(T : tableau[1..n] d'entiers)**

**pour i de 1 à n-1 faire**

**pour j de 1 à n-i faire**

**si  $T[j] > T[j+1]$  alors**

**échanger  $T[j]$  et  $T[j+1]$**

**fin si**

**fin pour**

**fin pour**

**Fin Algorithme**

# Tri à bulles

**ALGORITHME** *Bulles*

**VAR** x, i, n : entier    T : tableau [1..100] d'entier    échange : booléen

**DEBUT**

*{saisie de n et remplissage du tableau T par n valeurs entières quelconques}*

**répéter**

échange  $\leftarrow$  faux

**pour** i de 1 à n-1 **faire**

**si** (T[i] > T[i+1]) **alors**

    x  $\leftarrow$  T[i+1]

    T[i+1]  $\leftarrow$  T[i]

    T[i]  $\leftarrow$  x

    échange  $\leftarrow$  vrai

**FinSi**

**FinPour**

**jusqu'à** (échange = faux)

**Fin**

# Tri à bulles

## Exemple de tri à bulles

|                                     |                                                                              |    |    |    |    |    |                         |
|-------------------------------------|------------------------------------------------------------------------------|----|----|----|----|----|-------------------------|
| Tableau initial                     | <table><tr><td>26</td><td>14</td><td>2</td><td>0</td><td>7</td></tr></table> | 26 | 14 | 2  | 0  | 7  |                         |
| 26                                  | 14                                                                           | 2  | 0  | 7  |    |    |                         |
| Après la 1 <sup>ère</sup> itération | <table><tr><td>14</td><td>2</td><td>0</td><td>7</td><td>26</td></tr></table> | 14 | 2  | 0  | 7  | 26 | échange = vrai          |
| 14                                  | 2                                                                            | 0  | 7  | 26 |    |    |                         |
| Après la 2 <sup>ème</sup> itération | <table><tr><td>2</td><td>0</td><td>7</td><td>14</td><td>26</td></tr></table> | 2  | 0  | 7  | 14 | 26 | échange = vrai          |
| 2                                   | 0                                                                            | 7  | 14 | 26 |    |    |                         |
| Après la 3 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>7</td><td>14</td><td>26</td></tr></table> | 0  | 2  | 7  | 14 | 26 | échange = vrai          |
| 0                                   | 2                                                                            | 7  | 14 | 26 |    |    |                         |
| Après la 4 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>7</td><td>14</td><td>26</td></tr></table> | 0  | 2  | 7  | 14 | 26 | échange = faux<br>ARRÊT |
| 0                                   | 2                                                                            | 7  | 14 | 26 |    |    |                         |

# Tri par sélection (par minimum)

## *Principe :*

- Chercher l'indice du plus petit élément du tableau  $T[1..n]$  et permuter l'élément correspondant avec l'élément d'indice 1
- Chercher l'indice du plus petit élément du tableau  $T[2..n]$  et permuter l'élément correspondant avec l'élément d'indice 2
- .....
- Chercher l'indice du plus petit élément du tableau  $T[n-1..n]$  et permuter l'élément correspondant avec l'élément d'indice  $n-1$ ;

# Tri par sélection (par minimum)

|                                     |                                                                              |    |    |    |    |    |                                   |
|-------------------------------------|------------------------------------------------------------------------------|----|----|----|----|----|-----------------------------------|
| Tableau initial                     | <table><tr><td>26</td><td>14</td><td>2</td><td>0</td><td>7</td></tr></table> | 26 | 14 | 2  | 0  | 7  |                                   |
| 26                                  | 14                                                                           | 2  | 0  | 7  |    |    |                                   |
| Après la 1 <sup>ère</sup> itération | <table><tr><td>0</td><td>14</td><td>2</td><td>26</td><td>7</td></tr></table> | 0  | 14 | 2  | 26 | 7  | <b>i = 1</b><br><b>indmin = 4</b> |
| 0                                   | 14                                                                           | 2  | 26 | 7  |    |    |                                   |
| Après la 2 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>14</td><td>26</td><td>7</td></tr></table> | 0  | 2  | 14 | 26 | 7  | <b>i = 2</b><br><b>indmin = 3</b> |
| 0                                   | 2                                                                            | 14 | 26 | 7  |    |    |                                   |
| Après la 3 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>7</td><td>26</td><td>14</td></tr></table> | 0  | 2  | 7  | 26 | 14 | <b>i = 3</b><br><b>indmin = 5</b> |
| 0                                   | 2                                                                            | 7  | 26 | 14 |    |    |                                   |
| Après la 4 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>7</td><td>14</td><td>26</td></tr></table> | 0  | 2  | 7  | 14 | 26 | <b>i = 4</b><br><b>indmin = 5</b> |
| 0                                   | 2                                                                            | 7  | 14 | 26 |    |    |                                   |

# Tri par sélection (par minimum)

**ALGORITHME** *Selection*

**VAR** x, i, j, indmin, n : entier    T : tableau [1..100] d'entier

**DEBUT**

*{saisie de n et remplissage du tableau T par n valeurs entières quelconques}*

**pour** i de 1 à n-1 **faire**

indmin  $\leftarrow$  i

**pour** j de 1 + i à n **faire**

**si** ( T[j] < T[indmin] ) **alors**

        indmin  $\leftarrow$  j

**finsi**

**finpour**

x  $\leftarrow$  T[i]

T[i]  $\leftarrow$  T[indmin]

T[indmin]  $\leftarrow$  x

**FinPour**

**FIN**

*Recherche de indmin dans  
le tableau T[i..n]*

*Permutation de T[i] et T[indmin]*

# Tri par insertion

## *Principe*

Cette méthode consiste à prendre les éléments de la liste un par un et insérer chacun dans sa bonne place de façon que les éléments traités forment une sous-liste triée

- Comparer et permuter si nécessaire  $T[1]$  et  $T[2]$  de façon à placer le plus petit dans la case d'indice 1
- Comparer et permuter si nécessaire l'élément  $T[3]$  avec ceux qui le précèdent dans l'ordre ( $T[2]$  puis  $T[1]$ ) afin de former une sous-liste triée  $T[1..3]$
- ...
- Comparer et permuter si nécessaire l'élément  $T[n]$  avec ceux qui le précèdent dans l'ordre ( $T[n-1]$ ,  $T[n-2]$ , ...) afin d'obtenir un tableau trié

# Tri par insertion

**ALGORITHME** *insertion*

**VAR**  $x, i, j, \text{pos}, n$  : entier     $T$  : tableau [1..100] d'entier

**DEBUT**

*{saisie de n et remplissage du tableau T par n valeurs entières quelconques}*

**pour**  $i$  **de** 2 **à**  $n$  **faire**

$x \leftarrow T[i]$     // sauvegarde de  $T[i]$  dans une variable  $x$

$j \leftarrow i - 1$

**TantQue**  $j > 0$  et  $T[j] > x$  **Faire**

$T[j+1] \leftarrow T[j]$

$j \leftarrow j - 1$

**FinTantQue**

$T[j+1] \leftarrow x$     // Insertion de  $x$  à sa bonne position

**FinPour**

**Fin**

# Tri par insertion

## Principe

- Le tri par insertion est un algorithme de tri qui consiste à **parcourir un tableau en plaçant chaque élément à sa place définitive**, en le comparant aux éléments déjà triés
- Pour **chaque élément** du tableau, on le compare à tous les éléments qui le **précèdent**, jusqu'à trouver sa place
- Pour cela, on effectue des décalages successifs de tous les éléments plus grands que l'élément en cours, afin de créer une place libre pour y insérer l'élément

|                                     |                                                                              |    |    |    |    |    |        |
|-------------------------------------|------------------------------------------------------------------------------|----|----|----|----|----|--------|
| Tableau initial                     | <table><tr><td>26</td><td>14</td><td>2</td><td>0</td><td>7</td></tr></table> | 26 | 14 | 2  | 0  | 7  |        |
| 26                                  | 14                                                                           | 2  | 0  | 7  |    |    |        |
| Après la 1 <sup>ère</sup> itération | <table><tr><td>14</td><td>26</td><td>2</td><td>0</td><td>7</td></tr></table> | 14 | 26 | 2  | 0  | 7  | x = 14 |
| 14                                  | 26                                                                           | 2  | 0  | 7  |    |    |        |
| Après la 2 <sup>ème</sup> itération | <table><tr><td>2</td><td>14</td><td>26</td><td>0</td><td>7</td></tr></table> | 2  | 14 | 26 | 0  | 7  | x = 2  |
| 2                                   | 14                                                                           | 26 | 0  | 7  |    |    |        |
| Après la 3 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>14</td><td>26</td><td>7</td></tr></table> | 0  | 2  | 14 | 26 | 7  | x = 0  |
| 0                                   | 2                                                                            | 14 | 26 | 7  |    |    |        |
| Après la 4 <sup>ème</sup> itération | <table><tr><td>0</td><td>2</td><td>7</td><td>14</td><td>26</td></tr></table> | 0  | 2  | 7  | 14 | 26 | x = 7  |
| 0                                   | 2                                                                            | 7  | 14 | 26 |    |    |        |

# Recherche dichotomique

## *Principe*

- On veut chercher un élément dans un tableau **trié** dans le sens croissant  
Le but de cette recherche est de diviser l'intervalle de recherche par 2 à chaque itération

Pour cela, on procède de la façon suivante :

Soient premier et dernier les extrémités gauche et droite de l'intervalle dans lequel on cherche la valeur x.

On calcule m, l'indice de l'élément médian :

$$m = (\text{premier} + \text{dernier}) \text{ div } 2$$

# Recherche dichotomique

## *Principe*

Il y a trois cas possibles :

- $x < T[m]$  : l'élément  $x$ , s'il existe, se trouve dans l'intervalle [premier...  $m-1$ ]
- $x > T[m]$  : l'élément de valeur  $x$ , s'il existe, se trouve dans l'intervalle [ $m+1$ ... dernier]
- $x = T[m]$  :  $x$  est trouvé, la recherche est terminée

La recherche dichotomique consiste à itérer ce processus jusqu'à ce que l'on trouve  $x$  ou que l'intervalle de recherche soit vide

# Recherche dichotomique

## **Données**

T: Tab

x, premier, dernier, m: entier

trouve : booléen

## **Traitement :**

premier  $\leftarrow$  1

dernier  $\leftarrow$  n

trouve  $\leftarrow$  faux

## **Répéter**

m  $\leftarrow$  (premier + dernier) div2

**Si**(x<T[m]) **Alors**

dernier  $\leftarrow$  m-1

**Sinon**

**Si**(x>T[m]) **alors**

premier  $\leftarrow$  m+1

**Sinon**

trouve  $\leftarrow$  vrai

**Finsi**

**Finsi**

**Jusqu'à** (trouve=vrai ou premier > dernier)

**Si** (trouve) **Alors**

afficher("indice ", m)

**Sinon**

afficher(" introuvable ")

**Finsi**

## Exercice-1

Écrire une fonction `Fusion(T1, n1, T2, n2)` qui retourne un tableau `T3` contenant tous les éléments de `T1` et `T2` **dans l'ordre croissant**.

# Exercice-1 Correction

**Fonction** Fusion(T1, n1, T2, n2 : Entier) : Tableau[1..n1+n2] d'Entiers

**Début**

**Var** T3 : Tableau[1..n1+n2] d'Entiers

**Var** i, j, k : Entier

$i \leftarrow 1 ; j \leftarrow 1 ; k \leftarrow 1$

**// Étape 1 : comparaison parallèle**

**TantQue**  $i \leq n1$  **ET**  $j \leq n2$  **Faire**

**Si**  $T1[i] < T2[j]$  **Alors**

$T3[k] \leftarrow T1[i]$

$i \leftarrow i + 1$

**Sinon**

$T3[k] \leftarrow T2[j]$

$j \leftarrow j + 1$

**FinSi**

$k \leftarrow k + 1$

**FinTantQue**

**// Étape 2 : copier le reste de T1 (si non vide)**

**TantQue**  $i \leq n1$  **Faire**

$T3[k] \leftarrow T1[i]$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

**FinTantQue**

**// Étape 3 : copier le reste de T2 (si non vide)**

**TantQue**  $j \leq n2$  **Faire**

$T3[k] \leftarrow T2[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

**FinTantQue**

**Retourner** T3

**Fin**

## Exercice-2

Écrire un algorithme qui place tous les nombres **pairs** d'un tableau avant les **impairs**, sans trier.

# Exercice 2--correction

## Algorithme SéparerPairsImpairs

### Variables

T : tableau d'entiers

TF : tableau d'entiers (temporaire)

n, i, p, imp : entiers

### Début

p ← 1      // position pour les pairs dans TF

imp ← n     // position pour les impairs dans TF

**Pour** i ← 1 **à** n **faire**

**Si** T[i] % 2 = 0 **alors**

        TF[p] ← T[i]

        p ← p + 1

**Sinon**

        TF[imp] ← T[i]

        imp ← imp - 1

**FinSi**

**FinPour**

**Fin**

# Tableaux à deux dimensions

- Un tableau multidimensionnel est un tableau qui va lui-même contenir d'autres tableaux en valeurs

## Déclaration d'un tableau à deux dimensions

### **Variables**

Matrice : Tableau [1..3,1..4] de Réel

### **Types**

Matrice : Tableau [1..3,1..4] de Réel

### **Variables**

mat : Matrice

# Tableaux à deux dimensions

## Exemples

M1 : Tableau [1..30, 1..30] d'entier

M2 : Tableau [1..20, 1..20] de réel

M3 : Tableau [1..20, 1..20] de caractère

## Appel :

$X \leftarrow \text{Tab}[2, 3]$

$\text{Tab}[2, 3]$  permet d'accéder à l'élément de la matrice qui se trouve à l'intersection de la ligne 2 et de la colonne 3

# Tableaux à deux dimensions

Les tableaux à deux dimensions sont appelés des Matrices

**Tab**

|   | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 1 | 12 | 28 | 44 | 2  | 76 | 77 | 32 |
| 2 | 23 | 36 | 51 | 11 | 38 | 54 | 25 |
| 3 | 43 | 21 | 55 | 67 | 83 | 41 | 69 |

- C'est une matrice de 3 lignes et 7 colonnes de valeurs entiers
- Les éléments du tableau sont repérés par leur numéro de ligne et leur numéro de colonne

**Tab[2][5] = 38**

# Tableaux à deux dimensions

- Considérons un tableau M de L lignes et C colonnes:
  - Les indices du tableau varient de 1 à L, respectivement de 1 à C
  - La composante de la Nième N<sup>ième</sup> ligne et M<sup>ième</sup> colonne est notée : A[N,M]
- Pour accéder à un élément d'un tableau à deux dimensions, on écrit:

Nom\_tableau[Numéro\_ligne, Numéro\_Colonne]

# Tableaux à deux dimensions

## Algorithme InitTableau

Var

i,j: entier

Tab: tableau [1..100,1..50] de entier

Début

Pour i de 1 à 100 faire

    pour j de 1 à 50 faire

        Tab[i,j] ← 0

    Fin pour

Fin pour

Fin

## Algorithme AfficheTableau

Var

i,j: entier

Tab: tableau [1..100,1..50] de entier

Début

Pour i de 1 à 100 faire

    pour j de 1 à 50 faire

        ecrire(Tab[i,j])

    Fin pour

Fin pour

Fin

## Exercice – 1

1. Ecrire une procédure permettant de remplir une matrice
2. Ecrire une procédure permettant d'afficher une matrice
3. Ecrire une procédure permettant de calculer la somme de deux matrices M1 et M2
4. Ecrire une procédure permettant d'appeler les procédures précédentes